

Código de Golay extendido (24, 12) — Parte C

Trabajo Práctico 2 · Curso de FEC 2026 · Fundación Fulgor

Villar, Pedro

El RTL está en `rtl/`, un archivo por módulo, con las interfaces del enunciado. Los testbenches están en `tb/` y se corren con `cocotb` e `Icarus` desde `sim/` (`make todos`).

1 Ejercicio 8 — Codificador

`golay_mult_b` calcula $x \cdot B$ como el XOR de las filas b_i donde $x_i = 1$, igual que en la cuenta a mano de la Parte A. Cada fila se enmascara con el bit correspondiente y se hace el XOR de las doce:

```
o_vec = ({12{i_vec[11]}} & 12'h98F) ^ ... ^ ({12{i_vec[0]}} & 12'hEA9)
```

`popcount12` suma los doce bits en un `for` dentro de un `always_comb`.

`golay_encoder` es una instancia de `golay_mult_b` que calcula la paridad y un registro que guarda `{i_msg, paridad}`. Latencia de un ciclo.

2 Ejercicio 9 — Decodificador

Los cuatro submódulos son combinacionales:

- `golay_syndrome`: una instancia de `golay_mult_b` sobre $r[23:12]$ y un XOR con $r[11:0]$.
- `golay_row_search`: arma los doce candidatos $v \oplus b_i$ en paralelo con un `generate`, cada uno con su `popcount12`. Para no escribir la matriz de nuevo, cada fila b_i se obtiene pasando u_i por un `golay_mult_b`; como la entrada es constante, eso se reduce a cables. Después un `extttfor` recorre los candidatos del 0 al 11 y se queda con el primero de peso ≤ 2 , que es el de menor índice.
- `golay_err_gen`: un `if/else if` con los cuatro casos en orden. En el caso 5 levanta `o_uncorrectable` y deja el error en cero.
- `golay_correct`: $r \oplus e$, se queda con los 12 bits de arriba como mensaje, y `o_corrected` es el OR de los bits de e .

El decodificador une todo en tres etapas:

etapa	lógica	registros
E1	<code>golay_syndrome</code>	s, r
E2	$q = s \cdot B$, pesos y búsquedas de s y q	$s, q, w(s), w(q)$, las dos búsquedas, r
E3	<code>golay_err_gen, golay_correct</code>	salidas

En E2 también se registra s , que no figura en la tabla del enunciado, porque `golay_err_gen` lo necesita para el caso 1.

3 Ejercicio 10 — Verificación y análisis

Cada testbench toma el estímulo de `golay/stimulus.py` (Parte B) y compara la salida del RTL contra el modelo:

módulo	vectores	qué se verifica
<code>golay_mult_b</code>	4096	salida, y aplicado dos veces vuelve a la entrada ($B^2 = I$)
<code>popcount12</code>	4096	barrido exhaustivo
<code>golay_row_search</code>	4096	barrido exhaustivo
<code>golay_syndrome</code>	6420	síndrome nulo en las 4096 palabras código, más errores
<code>golay_err_gen</code>	15826	patrón de error y bandera
<code>golay_correct</code>	15826	palabra, mensaje y <code>o_corrected</code>
<code>golay_encoder</code>	4096	los 4096 mensajes, uno por ciclo
<code>golay_decoder</code>	15826	mensaje y error bit a bit, y las dos banderas

Pasan los ocho. En `golay_row_search`, cuando no hay fila, `o_idx` y `o_res` no se comparan, y en el decodificador `o_msg` y `o_err` no se comparan cuando está `o_uncorrectable`. Los vectores del decodificador pasan por los cinco casos: 5586, 4740, 1430, 3960 y 110.

Latencia. El codificador tiene latencia 1 y el decodificador 3, los dos con una palabra por ciclo. Los testbenches meten un vector por ciclo y comparan la salida 1 y 3 ciclos después; si la latencia fuera otra, fallarían.

Instancias. Dentro del decodificador:

submódulo	directas	dentro de otros	total
<code>golay_syndrome</code>	1	—	1
<code>golay_row_search</code>	2	—	2
<code>golay_err_gen</code>	1	—	1
<code>golay_correct</code>	1	—	1
<code>golay_mult_b</code>	1	1 en el síndrome, 12 en cada búsqueda	26
<code>popcount12</code>	2	12 en cada búsqueda	26

El codificador tiene una sola instancia de `golay_mult_b`. De las 26 del decodificador, 24 son las de las búsquedas, que tienen entrada constante y no generan lógica.